

```
In [ ]: #1.Difference between list and Tuples are:
1.Mutability:
List: Lists are mutable, meaning their contents (elements) can be modified after creation. You can add, remove, or
Tuple: Tuples are immutable, meaning once a tuple is created, its elements cannot be modified, added, or removed.
2.Syntax:
List: Lists are defined using square brackets [].
Tuple: Tuples are defined using parentheses (). Although tuples with a single element require a trailing comma.
3.Performance:
List: Lists have more overhead due to their mutability, so they are typically slower for certain operations (like i
Tuple: Since tuples are immutable, they are generally faster than lists for certain operations and use less memory.
```

```
In [1]: #2.
l1 = [1, 2, 3, 4, 5, 6]
l2 = [6, 5, 4, 3, 2, 1]
for val1, val2 in zip(l1, l2):
    if val1 > val2:
        print("L1 element value is greater than L2 element value")
    elif val1 < val2:
        print("L1 element value is less than L2 element value")
    else:
        print("L1 element value is equal to L2 element value")
```

```
L1 element value is less than L2 element value
L1 element value is less than L2 element value
L1 element value is less than L2 element value
L1 element value is greater than L2 element value
L1 element value is greater than L2 element value
L1 element value is greater than L2 element value
```

```
In [5]: #3
def numbers_to_squares():

    user_input = input("Enter numbers separated by spaces: ")
    num_list = list(map(int, user_input.split()))

    square_dict = {num: num ** 2 for num in num_list}

    return square_dict
```

```
result = numbers_to_squares()
print(result)
```

{2: 4, 4: 16, 8: 64}

In [38]: #4

```
def Calculator(int1, int2, task):
    task = task.lower()
    if task == 'add':
        return int1 + int2
    elif task == 'sub':
        return int1 - int2
    elif task == 'mul':
        return int1 * int2
    elif task == 'div':
        if int2 != 0:
            return int1 / int2
        else:
            return "Error: Division by zero is not allowed."
    else:
        return "Invalid task. Please provide 'add', 'sub', 'mul', or 'div'."

print(Calculator(2, 5, 'ADD'))
print(Calculator(7, 1, 'SuB'))
print(Calculator(10, 0, 'div'))
```

7

6

Error: Division by zero is not allowed.

In [12]: #5.

```
def is_leap_year(year):
    if (year % 4 == 0 and year % 100 != 0) or (year % 400 == 0):
        return True
    else:
        return False

def print_leap_years_21st_century():
    print("Leap years in the 21st Century (2001-2100):")
    for year in range(2001, 2101):
```

```
    if is_leap_year(year):  
        print(year)  
print_leap_years_21st_century()
```

Leap years in the 21st Century (2001–2100):

2004
2008
2012
2016
2020
2024
2028
2032
2036
2040
2044
2048
2052
2056
2060
2064
2068
2072
2076
2080
2084
2088
2092
2096

```
In [14]: #6.  
def print_pattern():  
    for i in range(1, 6):  
        for j in range(1, i + 1):  
            print(j, end=" ")  
        print()  
print_pattern()
```

```
1
1 2
1 2 3
1 2 3 4
1 2 3 4 5
```

```
In [30]: #7.
def convert_to_dict(tuple_list):
    return dict(tuple_list)
input_list = [("name", "Virat"), ("surname", "Kohli"), ("yob", 1986)]
output_dict = convert_to_dict(input_list)
print(output_dict)
```

```
{'name': 'Virat', 'surname': 'Kohli', 'yob': 1986}
```

```
In [36]: #8.
def split_even_odd_indexed_lists():
    user_input = input("Enter a list of numbers separated by commas: ")
    num_list = [int(num.strip()) for num in user_input.split(',')] # Convert the input string into a list of integers

    even_indexed = []
    odd_indexed = []

    for index, value in enumerate(num_list):
        if index % 2 == 0:
            even_indexed.append(value)
        else:
            odd_indexed.append(value)

    return even_indexed, odd_indexed
even_list, odd_list = split_even_odd_indexed_lists()
print("Even-indexed numbers:", even_list)
print("Odd-indexed numbers:", odd_list)
```

```
Even-indexed numbers: [1, 3, 7]
Odd-indexed numbers: [4, 2, 0]
```

```
In [22]: #9.
def longest_word(sentence):
    words = sentence.split()
    longest = words[0] if words else ""
```

```
    for word in words:
        if len(word) > len(longest):
            longest = word

    return longest
sentence = input("Enter a sentence: ")
print("The longest word is:", longest_word(sentence))
```

The longest word is: programming

```
In [40]: #10
def count_frequency(lst):
    frequency_dict = {}
    for item in lst:
        if item in frequency_dict:
            frequency_dict[item] += 1
        else:
            frequency_dict[item] = 1

    return frequency_dict
input_list = [1, 2, 2, 3, 4, 4, 4, 5, 5]
output = count_frequency(input_list)
print(output)
```

{1: 1, 2: 2, 3: 1, 4: 3, 5: 2}

In []: